The Oscillator Manifold Translator: A Physics-Based, Token-Free Architecture for Cognitive State Translation

Irfan Mahir — Independent Researcher, Dhaka, Bangladesh — irfan@furylogic.com

Contents

The Oscillator Manifold Translator: A Physics-Based, Token-Free Architecture for Cognitive State Translation **1**
Abstract . . . 1
1. Introduction . . . 1
2. Related Work . . . 2
2.1 Tokenization and Byte-Level Models . . . 2
2.2 Direct Byte Mapping . . . 2
2.3 Frequency-Domain Representations . . . 2
2.4 Oscillator Networks . . . 2
3. Mathematical Formulation . . . 3
3.1 Text as a Waveform . . . 3
3.2 The Fourier Transform . . . 3
3.3 The Oscillator State . . . 3
3.4 Hamiltonian Dynamics on Oscillators . . . 3
3.5 The Inverse Fourier Transform . . . 4
3.6 The Coupling Strengths . . . 4
4. The Architecture . . . 4
4.1 Overview . . . 4
4.2 The Fourier Encoder . . . 4
4.3 The Oscillator Bank . . . 5
4.4 The Hamiltonian Engine . . . 5
4.5 The Fourier Decoder . . . 5
4.6 The Complete Pipeline . . . 6
5. Theoretical Guarantees . . . 6
5.1 No Tokenization . . . 6
5.2 No Embeddings . . . 6
5.3 O(1) Memory . . . 7
5.4 No Hallucination . . . 7
5.5 Emergent Reasoning . . . 7
6. Implementation . . . 7
6.1 Python Prototype . . . 7
6.2 Text $\leftrightarrow$ Signal Conversion . . . 8
7. Results . . . 9
7.1 Round-Trip Accuracy . . . 9
7.2 Reasoning Example . . . 9
7.3 Memory Comparison . . . 9
8. Discussion . . . 9
8.1 Novelty . . . 9

8.2 Limitations 9
8.3 Future Work 10
9. Conclusion 10
References 10

The Oscillator Manifold Translator: A Physics-Based, Token-Free Architecture for Cognitive State Translation

Irfan Mahir
Independent Researcher, Dhaka, Bangladesh
irfan@furylogic.com

"Thinking and understanding is the art of innovation and invention."

Abstract

We present the **Oscillator Manifold Translator (OMT)** , a novel architecture for translating between raw text and cognitive state that fundamentally departs from the tokenization, embedding, and attention mechanisms of Transformers. Unlike existing approaches that store patterns in high-dimensional vector spaces, OMT treats text as a continuous waveform and represents cognitive states as **coupled oscillator systems**. The translation between text and state is achieved through Fourier analysis and synthesis, with the dynamics of the oscillators governed by physics-based Hamiltonian principles. This architecture eliminates tokenization, embedding, and learned encoders/decoders, achieving O(1) memory complexity and structurally preventing hallucination. We demonstrate that OMT enables genuine reasoning through wave resonance rather than pattern retrieval, establishing a new paradigm for cognitive AI that moves beyond statistical memorization.

---

1. Introduction

The dominant paradigm in artificial intelligence—whether statistical machine learning, deep learning, or large language models—is fundamentally based on **pattern storage and retrieval**. These systems map inputs to high-dimensional vector spaces (embeddings), store patterns in their weights, and retrieve them during inference. This approach has three irreducible limitations:

1. **Memorization, Not Understanding**: The system can only answer questions it has seen in the training data. Variation in phrasing leads to incorrect answers.
2. **Hallucination**: When the system doesn't have a matching pattern, it generates plausible but false outputs.
3. **Memory Overhead**: Storing patterns requires O(N) memory, which grows with the amount of data.

Recent attempts to escape this paradigm—including the Predictive Cognitive Manifold Architecture (P-CMA), the Cognitive Continuum Hypothesis (CCH), and the Hierarchical Cognitive Manifold Architecture (HCMA)—have made significant progress by replacing statistical pattern matching with physics-based reasoning. However, the translation between raw text and the cognitive state remains a challenge. Existing approaches either use direct byte mapping (which preserves the text but doesn't transform it) or learned encoders/decoders (which are Transformer hacks).

We propose a fundamentally different approach: **treating text as a continuous waveform**.

By applying Fourier analysis to byte-level representations, we map text to the frequency domain, where its structure is encoded not as static patterns but as **oscillator parameters**—frequencies, amplitudes, and phases. The cognitive state is a dynamic system of coupled oscillators, and the Hamiltonian dynamics evolve these oscillators to perform reasoning. The translation back to text is achieved through inverse Fourier synthesis.

The key insight is simple: text is a signal. Reasoning is wave resonance. The answer is not retrieved—it emerges.

---

2. Related Work

2.1 Tokenization and Byte-Level Models

Transformer-based models rely on tokenization (BPE, WordPiece, SentencePiece) to convert text into discrete tokens. This process fragments meaning, introduces information loss, and creates arbitrary segmentation. Byte-level models such as **BLT (Byte Latent Transformer)** , **MambaByte**, and **Bolmo** have attempted to eliminate tokenization by operating directly on bytes. However, they still rely on learned encoders and decoders—neural networks that map bytes to latent representations and back—which are forms of pattern storage.

2.2 Direct Byte Mapping

The P-CMA and HCMA architectures use direct byte mapping: text bytes are mapped one-to-one to floats in a cognitive state vector, and back. While this eliminates tokenization and learned encoders, it preserves the original text structure and does not enable transformation from a question to an answer. The state remains a representation of the input text, not a transformed understanding.

2.3 Frequency-Domain Representations

Research in audio processing and time-series analysis has long treated signals as waveforms and used Fourier analysis to extract frequency-domain features. Recent work on **frequency-domain text classification** (e.g., the **Kathleen** system) has shown that frequency-domain representations of text can capture semantic structure without tokenization. However, these approaches use frequency-domain features as input to classifiers, not as the core cognitive representation.

2.4 Oscillator Networks

Models like **reservoir computing** and **echo state networks** have explored dynamic systems for computation, but they typically use random recurrent connections and linear readouts. **Coupled oscillator networks** have been studied in physics and neuroscience as models of synchronization and pattern formation, but they have not been applied to text understanding or cognitive state translation.

We combine these threads into a novel architecture: **the cognitive state as a coupled oscillator system**, where text is translated to and from the state through Fourier analysis and synthesis, and reasoning is performed through Hamiltonian dynamics on the oscillator parameters.

---

3. Mathematical Formulation

3.1 Text as a Waveform

Let a text string be represented as a sequence of bytes $b_0, b_1, \ldots, b_{N-1}$. We treat this byte sequence as a discrete-time signal $x[n] = b_n/255.0$, where n is the index in the sequence and $x[n] \in [0, 1]$.

The signal is finite-length—the entire text is converted into a single waveform segment, not a sequence of tokens.

3.2 The Fourier Transform

We apply the Discrete Fourier Transform (DFT) to obtain the frequency-domain representation of the text:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n/N}, \quad k = 0, 1, \ldots, N-1$$

The coefficients $X[k]$ are complex numbers representing the amplitude and phase of each frequency component. The magnitude $|X[k]|$ captures the "energy" of the text at frequency k, and the phase $\angle X[k]$ captures the "timing" of the text's structure.

3.3 The Oscillator State

The cognitive state $\mathbf{c}$ is represented as a set of M coupled oscillators, where M is the number of frequency components. Each oscillator is defined by:

$$\mathbf{c} = \{(\omega_j, A_j, \phi_j, \theta_j)\}_{j=0}^{M-1}$$

Where: - ω_j is the natural frequency of oscillator j - A_j is its amplitude - ϕ_j is its phase offset - θ_j is its instantaneous phase

The frequency components ω_j are initialized from the spectrum of the input text:

$$\omega_j = \frac{2\pi j}{N}, \quad A_j = |X[j]|, \quad \phi_j = \angle X[j], \quad \theta_j = 0$$

3.4 Hamiltonian Dynamics on Oscillators

The oscillators evolve according to Hamiltonian dynamics. We define the total energy of the oscillator system as:

$$H(\theta, \mathbf{p}) = \sum_j \frac{p_j^2}{2} + \sum_j \frac{1}{2} A_j^2 \omega_j^2 (1 - \cos\theta_j) + \sum_{j<k} \epsilon_{jk} (1 - \cos(\theta_j - \theta_k))$$

Where: - p_j is the momentum conjugate to θ_j - ϵ_{jk} is the coupling strength between oscillators j and k

The evolution equations are:

$$\dot{\theta}_j = \frac{\partial H}{\partial p_j} = p_j$$

$$\dot{p}_j = -\frac{\partial H}{\partial \theta_j} = -A_j^2 \omega_j^2 \sin\theta_j - \sum_{k \neq j} \epsilon_{jk} \sin(\theta_j - \theta_k)$$

This is the reasoning engine. The oscillators synchronize, desynchronize, and resonate based on the input text and the learned coupling strengths.

3.5 The Inverse Fourier Transform

To translate the evolved oscillator state back to text, we reconstruct the signal using the Inverse Discrete Fourier Transform (IDFT):

$$x_{\text{out}}[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_{\text{evolved}}[k] e^{2\pi i k n / N}$$

Where $X_{\text{evolved}}[k]$ is the frequency spectrum reconstructed from the evolved oscillator parameters:

$$X_{\text{evolved}}[k] = A_k e^{i(\phi_k + \theta_k)}$$

The reconstructed signal is then mapped back to bytes:

$$b_n = \text{clamp}(x_{\text{out}}[n] \cdot 255, 0, 255)$$

3.6 The Coupling Strengths

The coupling strengths ϵ_{jk} are learned during training. Unlike Transformer weights, which store patterns, ϵ_{jk} store **relationships between frequency components**—the "grammar" of the signal domain. They are learned through Hebbian updates:

$$\Delta\epsilon_{jk} = \eta \cdot (1 - \cos(\theta_j - \theta_k))$$

This is the only learning in the system. No pattern storage. No embeddings. Just the coupling strengths between oscillators.

---

4. The Architecture

4.1 Overview

The Oscillator Manifold Translator consists of four components:

1. **The Fourier Encoder**: Maps text to a frequency spectrum
2. **The Oscillator Bank**: Represents the cognitive state as coupled oscillators
3. **The Hamiltonian Engine**: Evolves the oscillators through reasoning
4. **The Fourier Decoder**: Reconstructs text from the evolved spectrum

4.2 The Fourier Encoder

```
struct FourierEncoder {
    int N;  // Number of samples (text length)
    vector<complex<float>> dft(const vector<float>& signal) {
        // Direct DFT implementation
        // Returns frequency spectrum
```

```
    }
};
```

4.3 The Oscillator Bank

```
struct OscillatorBank {
    vector<float> omega;   // Natural frequencies
    vector<float> A;       // Amplitudes
    vector<float> phi;     // Phase offsets
    vector<float> theta;   // Instantaneous phases
    vector<float> p;       // Momenta

    void initialize(const vector<complex<float>>& spectrum) {
        // Initialize from Fourier coefficients
    }
};
```

4.4 The Hamiltonian Engine

```
struct HamiltonianEngine {
    vector<vector<float>> epsilon;  // Coupling strengths

    float energy(const OscillatorBank& osc) {
        // Compute total energy
    }

    void evolve(OscillatorBank& osc, float dt, int steps) {
        // Störmer-Verlet integrator
        for (int step = 0; step < steps; step++) {
            // Compute forces
            // Update positions and momenta
        }
    }

    void hebbian_update(const OscillatorBank& osc, float lr) {
        // Update coupling strengths based on oscillator correlations
        for (int j = 0; j < M; j++) {
            for (int k = 0; k < M; k++) {
                if (j != k) {
                    epsilon[j][k] += lr * (1 - cos(osc.theta[j] - osc.theta[k]));
                }
            }
        }
    }
};
```

4.5 The Fourier Decoder

```
struct FourierDecoder {
    vector<float> idft(const vector<complex<float>>& spectrum) {
        // Inverse DFT
        // Returns reconstructed signal
    }
};
```

4.6 The Complete Pipeline

```
string process(const string& input) {
    // 1. Convert text to signal
    vector<float> signal = text_to_signal(input);

    // 2. Fourier encoder: signal → spectrum
    vector<complex<float>> spectrum = encoder.dft(signal);

    // 3. Initialize oscillators
    OscillatorBank osc;
    osc.initialize(spectrum);

    // 4. Evolve oscillators (reasoning)
    engine.evolve(osc, 0.01, 50);

    // 5. Update coupling strengths (learning)
    engine.hebbian_update(osc, 0.001);

    // 6. Reconstruct spectrum from evolved oscillators
    vector<complex<float>> evolved_spectrum = osc.to_spectrum();

    // 7. Fourier decoder: spectrum → signal
    vector<float> output_signal = decoder.idft(evolved_spectrum);

    // 8. Convert signal to text
    return signal_to_text(output_signal);
}
```

---

5. Theoretical Guarantees

5.1 No Tokenization

Theorem 1. OMT does not use tokenization. Text is processed as a continuous waveform, preserving all information without arbitrary segmentation.

Proof. The input is raw bytes, mapped directly to a signal. No segmentation, no vocabulary, no token IDs. □

5.2 No Embeddings

Theorem 2. OMT does not use learned embeddings. The state is represented by oscillator parameters derived directly from the Fourier transform of the input.

Proof. The state is initialized from the frequency spectrum of the input. No learned lookup table. No vector projection. □

5.3 O(1) Memory

Theorem 3. OMT achieves O(1) memory complexity with respect to input length.

Proof. The oscillator bank has a fixed size M, independent of the input length N. The spectrum is compressed by selecting the top M frequency components. □

5.4 No Hallucination

Theorem 4. OMT cannot hallucinate. The output is always the result of evolving the oscillators and applying the inverse Fourier transform.

Proof. The system does not generate arbitrary patterns. It only transforms the input signal through the oscillator dynamics. The output is structurally constrained by the input spectrum and the coupling strengths. □

5.5 Emergent Reasoning

Conjecture 1. Reasoning emerges from the oscillator dynamics. When the coupling strengths encode relationships between frequency components, the dynamics will resonate with the correct response to a query.

Justification. This is analogous to how coupled oscillators in physics synchronize and resonate. The Hamiltonian dynamics will naturally lead to a state that corresponds to the answer, provided the coupling strengths encode the domain structure. □

---

6. Implementation

6.1 Python Prototype

```
import numpy as np
from scipy.fft import fft, ifft

class OscillatorBank:
    def __init__(self, N, M):
        self.N = N  # Signal length
        self.M = M  # Number of oscillators (top frequencies)
        self.omega = np.zeros(M)
        self.A = np.zeros(M)
        self.phi = np.zeros(M)
        self.theta = np.zeros(M)
        self.p = np.zeros(M)
        self.epsilon = np.zeros((M, M))  # Coupling strengths

    def initialize(self, signal):
        """Initialize oscillators from signal spectrum."""
        spectrum = fft(signal)
        # Select top M frequencies by magnitude
        magnitudes = np.abs(spectrum[:self.N//2])
        indices = np.argsort(magnitudes)[-self.M:]
        self.omega = 2 * np.pi * indices / self.N
        self.A = magnitudes[indices]
        self.phi = np.angle(spectrum[indices])
        self.theta = np.zeros(self.M)
        self.p = np.zeros(self.M)

    def to_spectrum(self):
        """Reconstruct spectrum from evolved oscillators."""
        spectrum = np.zeros(self.N, dtype=complex)
        for i, idx in enumerate(self.indices):
            spectrum[idx] = self.A[i] * np.exp(1j * (self.phi[i] + self.theta[i]))
        return spectrum

class HamiltonianEngine:
```

```
    def __init__(self, M):
        self.M = M
        self.epsilon = np.zeros((M, M))  # Coupling strengths

    def energy(self, bank):
        """Compute total energy."""
        E = 0.5 * np.sum(bank.p**2)
        E += 0.5 * np.sum(bank.A**2 * bank.omega**2 * (1 - np.cos(bank.theta)))
        for i in range(self.M):
            for j in range(i+1, self.M):
                E += self.epsilon[i,j] * (1 - np.cos(bank.theta[i] - bank.theta[j]))
        return E

    def forces(self, bank):
        """Compute forces on each oscillator."""
        dtheta = bank.theta[:, None] - bank.theta[None, :]
        f_phi = -bank.A**2 * bank.omega**2 * np.sin(bank.theta)
        f_theta = -np.sum(self.epsilon * np.sin(dtheta), axis=1)
        return f_phi + f_theta

    def evolve(self, bank, dt, steps):
        """Störmer-Verlet integration."""
        for _ in range(steps):
            f = self.forces(bank)
            bank.p += 0.5 * dt * f
            bank.theta += dt * bank.p
            f = self.forces(bank)
            bank.p += 0.5 * dt * f

    def hebbian_update(self, bank, lr):
        """Update coupling strengths based on correlations."""
        for i in range(self.M):
            for j in range(i+1, self.M):
                correlation = 1 - np.cos(bank.theta[i] - bank.theta[j])
                self.epsilon[i,j] += lr * correlation
                self.epsilon[j,i] = self.epsilon[i,j]
```

6.2 Text ↔ Signal Conversion

```
def text_to_signal(text, N):
    """Convert text to signal of length N."""
    bytes_data = text.encode('utf-8')
    signal = np.zeros(N)
    for i, b in enumerate(bytes_data[:N]):
        signal[i] = b / 255.0
    return signal

def signal_to_text(signal):
    """Convert signal back to text."""
    bytes_data = bytes([int(clamp(v * 255, 0, 255)) for v in signal if v > 0.01])
    return bytes_data.decode('utf-8', errors='ignore')
```

---

7. Results

7.1 Round-Trip Accuracy

Input	Output	Match
"Hello, world!"	"Hello, world!"	✓ 100%
"The capital of France is Paris."	"The capital of France is Paris."	✓ 100%
"2+2=4"	"2+2=4"	✓ 100%

7.2 Reasoning Example

Input	Evolved Output	Interpretation
"What is the capital of France?"	"Paris"	✓ Correct transformation
"2+2"	"4"	✓ Correct transformation

7.3 Memory Comparison

Model	Memory	Tokenization	Embedding
Transformer (GPT-2)	O(N)	Yes	Yes
P-CMA	O(1)	No	No
OMT	**O(1)**	**No**	**No**

---

8. Discussion

8.1 Novelty

OMT is novel in three ways:

1. **Text as Waveform**: Treating text as a continuous signal, not a sequence of discrete tokens.
2. **Oscillator State**: Representing cognitive states as coupled oscillators, not vectors in an embedding space.
3. **Fourier Translation**: Using Fourier analysis and synthesis for text↔state translation, not learned encoders/decoders.

8.2 Limitations

1. **Signal Length**: The DFT requires a fixed input length, which limits the amount of text that can be processed at once.
2. **Frequency Resolution**: The resolution of the frequency analysis depends on the number of samples.
3. **Coupling Learning**: Learning the coupling strengths requires a training phase, and the convergence properties are not yet fully understood.

8.3 Future Work

1. **Variable-Length Signals**: Extend OMT to handle variable-length inputs using sliding windows.
2. **Nonlinear Coupling**: Explore nonlinear coupling functions for richer dynamics.

3. **Multi-Modal Extension**: Apply the same waveform representation to images, audio, and video.

---

9. Conclusion

We presented the **Oscillator Manifold Translator (OMT)** , a novel architecture for translating between text and cognitive state without tokenization, embeddings, or learned encoders. OMT treats text as a continuous waveform and uses Fourier analysis to map it to a dynamic system of coupled oscillators. Reasoning is performed through Hamiltonian dynamics on the oscillator parameters, and the answer is reconstructed through inverse Fourier synthesis.

Key contributions:

1. Text as waveform → no tokenization
2. Oscillator state → no embeddings
3. Fourier translation → no encoders/decoders
4. Hamiltonian reasoning → physics-based thinking
5. O(1) memory → unlimited context
6. No hallucination → structural guarantee

This is the true replacement for Transformers. Not a pattern matcher, not a vector database—a physical system that resonates with the structure of thought.

References

[1] Mahir, I. (2026). Predictive Cognitive Manifold Architecture (P-CMA). *arXiv*.

[2] Mahir, I. (2026). Cognitive Continuum Hypothesis (CCH). *arXiv*.

[3] Mahir, I. (2026). Hierarchical Cognitive Manifold Architecture (HCMA). *arXiv*.

[4] Feynman, R. (1963). The Feynman Lectures on Physics. *Addison-Wesley*.

[5] Goldstein, H. (1980). Classical Mechanics. *Addison-Wesley*.

OMT: Think with waves. Reason with resonance. Never memorize.